

CPCS203 Programming-II Spring 2026

Library Borrowing & Fine Management System

Assignment 1

Assigned Date: Sunday 15/2/2026

Due Date: Saturday, 28/3/2026 at 11 P.M.

Instructions:

- **Submission Platform:** Submit the program exclusively on Blackboard.
- Ensure you **include your name, ID, section, and assignment number** at the beginning of your program.
- **Weight:** This assignment is worth 10% of your overall grade.
- **Deadline:** No submissions will be accepted after 11:00 PM for any reason.

Assignment Objective

Design and implement a **Library Borrowing System** that helps librarians manage:

- Books
- Library members
- Borrowing rules & fine policies
- Issued borrowing records
- Fine calculations and summary reports

This assignment focuses on:

- Multidimensional & ragged arrays
- Classes and objects
- File I/O (read/write)
- Array processing
- Object interaction

System Description

The Library Borrowing System manages book borrowing activities and calculates fines based on borrowing duration and rule violations.

The system should:

1. Register all books, members, and fine rules from `input.txt`
2. Store them in arrays
3. Issue borrow records
4. Calculate fines (late return, damaged book, underage member)
5. Write all outputs to `LibraryDB.txt`

Files Used

1- Input File

- `input.txt`: contains:
 - Books
 - Members
 - Fine rules
 - Borrow command

2- Output File

- `LibraryDB.txt`: displays:
 - Registered data
 - Borrow records
 - Fine details
 - Total borrows per member

Step 1: Creating Arrays

Before issuing borrow requests, you need to create multiple arrays as described below.

1. **Books Array:** an array of Books (`Book[] books`) that stores all registered books. Its length is the first number found in the first line of `input.txt`. Each book has an ISBN, title, author, and published year.

Field	Type	Example
ISBN	String	9780134685991
Title	String	JavaProgramming
Author	String	Deitel

PublishYear	int	2019
-------------	-----	------

2. **Members Array:** an array of Members (`Member[] members`) that stores members' information, such as `memberID`, `firstName`, `lastName`, and `birthYear`. Its length is the second number found in the first line of `input.txt`.

Field	Type	Example
MemberID	String	2023123456
FirstName	String	Ahmed
LastName	String	Saleh
BirthYear	int	2003

3. **Fine Rules Array:** is a 2D ragged Array (`double[][] fineRules`) that stores the fine information such as type, fine levels, and amount. The array's number of rows (number of fine types) is the third number found in `input.txt` (i.e., the number on the second line in the input file). Each row length is the number of severity levels.

Fine Type	Levels	Example
LateReturn	3	10, 20, 50
DamagedBook	2	100, 300
LostBook	1	500

Example visualization:

Fine Type	Level 1	Level 2	Level 3
LateReturn	10	20	50
DamagedBook	100	300	-
LostBook	500	-	-

2. **Borrow Records Array:** is an array that stores borrow records (`BorrowRecord[] records`). Its length is the fourth number in the input file (i.e., the number on the third line in the input file).
3. **Fine Codes Array:** is an array of Strings that stores the fine types.

```
String[] fineCodes = {
    "LateReturn",
    "DamagedBook",
    "LostBook"
};
```

Step 2: Adding Data to the System

In this step, the system reads commands from the `input.txt` file and registers all required data into the appropriate arrays. Each command begins with a keyword that identifies the operation to be performed. The system processes the commands sequentially in the order they appear in the input file.

2.1 AddFineRule

This command is used to define the fine policy for a specific borrowing violation. Each fine rule is associated with a fine code and multiple severity levels, where each level has a corresponding fine amount.

- The system first searches the `fineCode` array to determine the index of the given fine code.
- A new row is then created in the `fineRules` array at that index.
- The length of the row depends on the number of severity levels specified in the command.
- Fine amounts are stored in the order of increasing severity.

Command Example
AddFineRule LateReturn 3 10 20 50

Example explanation:

- `LateReturn` → Fine code
- `3` → Number of severity levels
- `10, 20, 50` → Fine amounts for severity levels 1, 2, and 3, respectively

2.2 AddBook

This command registers a new book in the library system and stores it in the `Books` array.

- Each book is stored as an object of the `Book` class.
- The system reads the book details from the input file and assigns them to the object's attributes.
- Books are stored sequentially in the array in the order they appear in the input file.

Command Example
AddBook 9780134685991 JavaProgramming Deitel 2019

Example explanation:

- `9780134685991` → Unique identifier for the book

- `JavaProgramming` → Book title
- `Deitel` → Book author
- `2019` → Year the book was published

2.3 AddMember

This command registers a new library member and stores the information in the `Members` array.

- Each member is represented as an object of the `Member` class.
- The member's information is read from the input file and stored in the array.
- Member IDs are assumed to be unique and are later used to identify members when issuing borrow records.

Command Example
<code>AddMember 2023123456 Ahmed Saleh 2003</code>

Example explanation:

- `2023123456` → Unique identification number for the member
- `Ahmed` → Member's first name
- `Saleh` → Member's last name
- `2003` → Year of birth (used to calculate age)

Step 3: Issuing Borrow Records

3.1 IssueBorrow

This command creates a borrowing record and applies the corresponding fine if a rule violation occurs.

- The system should search the `Books` array using the ISBN to find the borrowed book.
- The system should search the `Members` array using the Member ID to find the borrower.
- The system should search the `fineCodes` array to locate the correct row in the `fineRules` array.
- The base fine is determined by the violation severity level.
- A unique borrow record number should be generated automatically.
- Additional fines may be applied based on predefined conditions.

Command Example
<code>IssueBorrow 9780134685991 2023123456 LateReturn 2</code>

Example explanation:

- `9780134685991` → Identifies the borrowed book

- 2023123456 → Identifies the borrower
- LateReturn → Type of borrowing violation
- 2 → Degree of violation severity

Fine Calculation Rules

1. Base Fine → from fineRules array
2. Underage Member (<18) → +100 SAR
3. Book Published Before 2000 → +50 SAR

Step 4: Output Format

After processing all commands from the input.txt file, the system generates a detailed report and writes it to the output file `LibraryDB.txt`. The output is organized in a structured and readable format to clearly present all borrowing and fine-related information.

4.1 Borrow Record Output

For every `IssueBorrow` command processed, the system prints a complete borrowing record. Each record represents one borrowing transaction and its associated fine details.

The printed information must include:

- **Borrow Record Information**
 - Borrow record number (automatically generated and unique)
 - Fine code associated with the borrowing violation
 - Severity level of the violation
 - Base fine amount retrieved from the fine rules array
- **Book Information**
 - ISBN number
 - Book title
 - Author name
 - Publish year
- **Member Information**
 - Member ID
 - Member full name
 - Birth year
- **Final Fine Calculation**
 - Total fine amount after applying all additional fines

Each borrow record must be clearly separated from the next one using a visual separator line to improve readability.

Example Output for "IssueBorrow 9781492056526 2022765432 LostBook 1"
--

```

===== Borrow Record Information =====
Record No= 12
Fine Code= LostBook
Severity= 1
Base Fine= 500.0

Book Information
ISBN= 9781492056526
Title= JavaOOP
Author= Ross
Publish Year= 1998

Member Information
ID= 2022765432
Name= Laila Hassan
Birth Year= 2010

Total Fine= 650.0

```

4.2 Borrow Summary Report (Borrows per Member) and “Quit” command

The `quit` command instructs the system to print a summary report showing the total number of borrowing records for each member, and end the reading process.

The summary report must include:

- Member ID
- Member full name
- Total number of borrows issued to the member

This report helps analyze borrowing activity per member and verifies correct record tracking.

Example Summary		
----- Total Borrow(s) Per Member -----		
Member ID	Name	Total Borrows
2023123456	Ahmed Saleh	4
2023987456	Sara Ali	3
2022456789	Omar Khaled	2
2022765432	Laila Hassan	3

4.3 Output Formatting Requirements

To ensure consistency and correctness:

- The output must be written only to `LibraryDB.txt`.

- The format, labels, spacing, and order of information must match the provided sample output.
- Each section should be clearly separated and easy to read.
- No extra text or debugging information should appear in the output file.
- Numeric values (such as fines) should be displayed in a readable numeric format.
- The output file should be overwritten each time the program runs.
- All issued borrow records must be printed in the same order as they appear in the input file.
- The summary report must appear after all borrow records are printed.

Important Notes:

- Use of class & object, arrays, and passing object to method
- Use of Files, Reading/Writing from/on files
- Your program output must be exactly same as the given sample output file.
- Your display should be in a readable form.
- Organize your code in separated methods.
- Document your code with comments.

Deliverables:

- You should submit one zip file containing all Java codes: **BA1887415_A1.java**, where BA is your section, 1887415 is your ID, and A1 is Assignment 1.
- NOTE: your name, ID, and section number should be included as comments in all files.

UML Class Diagram

Main LibrarySystem Class

Necessary arrays are variables

```
+ addBook(..)
+ addMember(..)
+ addFineRule(..)
+ generateBorrow(..)
+ printBorrow(..)
+ printBorrowsPerMember(..)
```

Book Class

```
- isbn: String
- title: String
- author: String
- publishYear: int
+ constructors
+ getters/setters
+ toString()
```

Member Class

- memberID: String
- firstName: String
- lastName: String
- birthYear: int
- + constructors
- + getters/setters
- + toString()

BorrowRecord Class

- recordNo: int
- book: Book
- member: Member
- fineCode: String
- severity: int
- baseFine: double
- + constructors
- + CalculateFinalFine(..)
- + toString()